

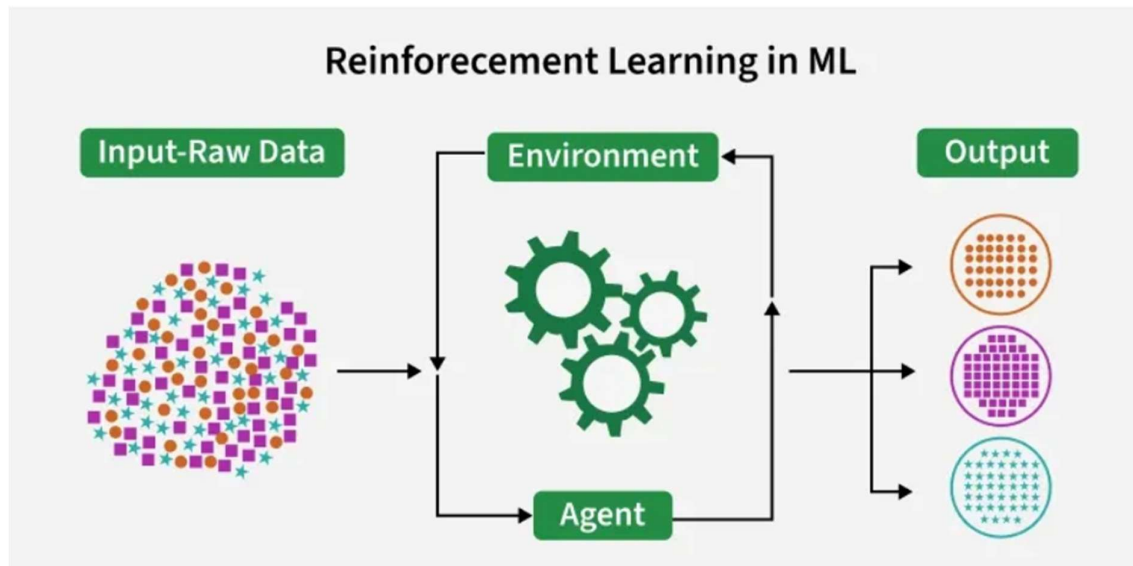
UNIT-V

Reinforcement Learning: Passive reinforcement learning, direct utility estimation, adaptive dynamic programming, temporal difference learning, active reinforcement learning- Q learning.

// By Manish D@nge

Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning that focuses on how agents can learn to make decisions through trial and error to maximize cumulative rewards. RL allows machines to learn by interacting with an environment and receiving feedback based on their actions. This feedback comes in the form of rewards or penalties.



Reinforcement Learning

Reinforcement Learning revolves around the idea that an agent (the learner or decision-maker) interacts with an environment to achieve a goal. The agent performs actions and receives feedback to optimize its decision-making over time.

- **Agent:** The decision-maker that performs actions.
- **Environment:** The world or system in which the agent operates.
- **State:** The situation or condition the agent is currently in.
- **Action:** The possible moves or decisions the agent can make.
- **Reward:** The feedback or result from the environment based on the agent's action.

Core Components

Let's see the core components of Reinforcement Learning

1. Policy

- Defines the agent's behavior i.e maps states for actions.
- Can be simple rules or complex computations.
- **Example:** An autonomous car maps pedestrian detection to make necessary stops.

2. Reward Signal

- Represents the goal of the RL problem.
- Guides the agent by providing feedback (positive/negative rewards).
- **Example:** For self-driving cars rewards can be fewer collisions, shorter travel time, lane discipline.

3. Value Function

- Evaluates long-term benefits, not just immediate rewards.
- Measures desirability of a state considering future outcomes.
- **Example:** A vehicle may avoid reckless maneuvers (short-term gain) to maximize overall safety and efficiency.

4. Model

- Simulates the environment to predict outcomes of actions.
- Enables planning and foresight.
- **Example:** Predicting other vehicles' movements to plan safer routes.

Working of Reinforcement Learning

The agent interacts iteratively with its environment in a feedback loop:

- The agent observes the current state of the environment.
- It chooses and performs an action based on its policy.
- The environment responds by transitioning to a new state and providing a reward (or penalty).
- The agent updates its knowledge (policy, value function) based on the reward received and the new state.

- This cycle repeats with the agent balancing exploration (trying new actions) and exploitation (using known good actions) to maximize the cumulative reward over time.

Types of Reinforcements

1. Positive Reinforcement: Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.

- **Advantages:** Maximizes performance, helps sustain change over time.
- **Disadvantages:** Overuse can lead to excess states that may reduce effectiveness.

2. Negative Reinforcement: Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.

- **Advantages:** Increases behavior frequency, ensures a minimum performance standard.
- **Disadvantages:** It may only encourage just enough action to avoid penalties.

Application

- **Robotics:** RL is used to automate tasks in structured environments such as manufacturing, where robots learn to optimize movements and improve efficiency.
- **Games:** Advanced RL algorithms have been used to develop strategies for complex games like chess, Go and video games, outperforming human players in many instances.
- **Industrial Control:** RL helps in real-time adjustments and optimization of industrial operations, such as refining processes in the oil and gas industry.
- **Personalized Training Systems:** RL enables the customization of instructional content based on an individual's learning patterns, improving engagement and effectiveness.

Advantages

- Solves complex sequential decision problems where other approaches fail.
- Learns from real-time interaction, enabling adaptation to changing environments.

- Does not require labeled data, unlike supervised learning.
- Can innovate by discovering new strategies beyond human intuition.
- Handles uncertainty and stochastic environments effectively.

Disadvantages

- Computationally intensive, requiring large amounts of data and processing power.
- Reward function design is critical; poor design leads to unintended behaviors.
- Not suitable for simple problems where traditional methods are more efficient.
- Challenging to debug and interpret, making it hard to explain decisions.
- Exploration-exploitation trade-off requires careful balancing to optimize learning.

Passive Reinforcement Learning in AI

A basic idea in artificial intelligence, Passive RL is the learning process of reaching a specific goal without doing exploratory actions unlike other RL techniques. The use of this form of RL may be applicable where the exploration is too expensive or poses certain risks and this is true in some branches of healthcare, finance and operations.

What does Passive Reinforcement Learning mean?

In the conventional reinforcement learning, then agents' purposefully search for possible state-action pairs in order to achieve the anticipated greatest cumulative rewards. However, in passive [reinforcement learning](#), the agent's task is simplified: it receives a fixed policy (a pre-determined list of actions) but it learns how to assign a value to circles stating whether or not to follow the policy. The focus shifts from learning the optimal actions to estimating the reward for adhering to a given strategy. This is often used in cases where policies are generated externally and exploring alternative actions might be too complex, dangerous, or unnecessary.

Key Features of Passive Reinforcement Learning

Here are key features of Passive Reinforcement Learning:

1. **Policy Evaluation Focus:** It evaluates a fixed policy rather than exploring new actions.
2. **State Value Estimation:** Uses state values to estimate rewards over time following a predefined policy.
3. **Bellman Equation:** Applies the Bellman Equation iteratively to refine reward estimates.
4. **Model-Based and Model-Free:** Supports both approaches; model-based uses environment dynamics, while model-free learns from real experiences.
5. **Temporal Difference Learning:** Updates state values in real-time based on reward discrepancies.
6. **Low-Risk Application:** Suitable for scenarios where exploration is costly or risky, such as finance and healthcare.

Key Concepts in Passive Reinforcement Learning

1. **Policy Evaluation:** Since the policy is fixed, the primary objective is to evaluate this policy's efficacy. Policy evaluation determines how much reward an agent can expect over time by following a particular policy from any given state. The agent learns to predict future rewards by calculating the state values—an estimation of the total reward from each state while following the policy.
2. **Bellman Equation for Evaluation:** Passive RL employs the Bellman Equation to estimate state values. This recursive formula calculates the value of a state based on the immediate reward and the value of future states. The agent iteratively applies this equation to refine its understanding of the rewards associated with different states until it reaches an optimal estimate for each state.
3. **Model-Based and Model-Free Approaches**
 - a. **Model-Based:** This approach assumes access to a model of the environment that includes the probabilities of transitioning between states and receiving rewards. In passive RL, this allows for dynamic programming techniques like policy iteration or value iteration.
 - b. **Model-Free:** Model-free passive RL doesn't require knowledge of the environment's dynamics. Instead, it learns from actual experiences using techniques like [Monte Carlo methods](#), which average out rewards from episodes, or Temporal Difference (TD) learning, which updates estimates based on discrepancies between consecutive steps.

Learning Methods in Passive RL

1. **Temporal Difference (TD) Learning:** TD learning is a popular approach in passive RL, especially in model-free scenarios. It updates the estimated state value after each interaction based on the difference between predicted and actual rewards (the temporal difference error). TD learning is particularly efficient because it can update values in real-time without waiting for an entire episode to conclude.
2. **Monte Carlo Methods:** Monte Carlo methods are episodic, meaning they wait until an episode ends to update state values based on accumulated rewards. This approach is less suitable for applications needing immediate feedback but is effective in deterministic or fully observable environments.

Some Applications of Passive RL

Passive reinforcement learning is widely applied in scenarios where agents must learn from observation or simulated experiences. For instance:

1. **Healthcare:** In fields like predictive diagnostics or medical decision support, passive RL can analyze historical data, learn the outcomes of fixed treatment strategies, and help improve future predictions.
2. **Finance:** Passive RL can model trading policies or risk management strategies by learning from historical market data without direct intervention in volatile environments.
3. **Robotics and Automation:** By assessing pre-determined paths, passive RL helps in quality control and improving precision in manufacturing where exploration is costly.

Passive RL vs. Active RL: A Comparison

Unlike active RL, where the agent seeks out the best actions, passive RL relies on predefined actions and focuses on understanding the value of these actions. While active RL is suitable for discovery and optimization, passive RL is ideal for controlled environments or settings with limited flexibility.

Conclusion

Passive Reinforcement Learning, by focusing on the evaluation of predefined strategies, offers a practical, safe, and resource-efficient way for agents to learn in stable, non-explorative environments. This method of learning holds promises for industries like healthcare, finance, and automated manufacturing, where exploration is costly or risky. By leveraging passive RL, AI systems can deliver insights and optimized decision-making while minimizing uncertainty.

Deep Explanation: Active vs Passive Reinforcement Learning

■ Passive Reinforcement Learning (Passive RL)

Passive RL is the simpler form of RL.

Here, the agent **does not decide its actions**. It is given a **fixed policy** (a rule that tells what action to take in each state).

The agent's only job is to **observe** what happens when it follows that policy and **learn the value of states**.

It does not try to improve the policy, and it does not explore.

Because actions are fixed, the agent simply collects experience to understand:

- how good each state is,
- what long-term reward it leads to,
- and how accurate the given policy is.

Methods like **Direct Utility Estimation, Temporal Difference learning, and Adaptive Dynamic Programming** are used in Passive RL.

■ Active Reinforcement Learning (Active RL)

Active RL is a more advanced and realistic form of RL.

Here, the agent **chooses its own actions**, explores the environment, and **tries to discover the best policy** by itself.

It must balance:

- **Exploration** → trying new actions
- **Exploitation** → choosing the best-known action

The agent learns both:

- the **optimal action for each state**
- the **long-term value of states**

Examples include **Q-learning, SARSA, Policy Gradients**, and **Deep RL** methods.

Active RL is used when no policy is given, and the agent must learn everything through trial and error.

DEEP Difference Table: Active RL vs Passive RL

Aspect	Passive Reinforcement Learning	Active Reinforcement Learning
Action Selection	Agent does not choose actions; follows fixed policy	Agent chooses actions based on learning
Policy	Fixed; given by the environment or designer	Learned from scratch by the agent
Goal	Evaluate how good the given policy is	Discover the optimal policy
Learning Focus	Learn state values	Learn both state values and optimal actions
Exploration	No exploration (actions pre-decided)	Requires exploration (ϵ -greedy, softmax, etc.)
Control	Passive observer; no control over behavior	Full control over its behavior
Environment Interaction	Only observes outcomes	Interacts actively, chooses, experiments
Computational Need	Lower—does not solve control problem	Higher—must solve control + prediction problems
Examples	Robot following a fixed path and evaluating rewards	Robot trying multiple paths to find the best one
Algorithms	Direct Utility Estimation, TD(0), ADP	Q-Learning, SARSA, Policy Gradients, Deep RL
When Used	When policy is known but we need values	When no policy is given and agent must learn everything
Uncertainty Handling	Does not handle uncertainty in actions	Must handle uncertainty and trade-offs

Aspect	Passive Reinforcement Learning	Active Reinforcement Learning
Optimality	Cannot find optimal policy	Designed to find optimal policy
Reward Usage	Feedback is used only for evaluation	Feedback is used for decision-making and improvement
Exploit/Explore Trade-off	Not required	Essential

Direct Utility Estimation in Artificial Intelligence

Direct Utility Estimation is a method used in **Reinforcement Learning (RL)**, particularly in **passive learning scenarios**, where the agent evaluates a fixed policy to estimate the utility of states. This approach involves running multiple trials or episodes, where the agent transitions through states and collects rewards until reaching a terminal state. The utility of a state is then estimated as the **average of the sample rewards** observed during these trials.

Key Characteristics

In Direct Utility Estimation, the agent assumes that the utilities of states are **independent** of each other. This means the utility of a state is calculated solely based on the rewards observed during the trials, without considering the relationship between states. While this simplifies the computation, it introduces a significant limitation because state utilities in RL are typically **Markovian**, meaning they depend on the utilities of successor states.

The utility of a state ($U(s)$) is estimated as: $[U(s) = \text{Average of observed rewards from state } s \text{ over multiple trials.}]$

Example

Consider a **4x3 grid environment** where an agent can move in four directions: Left, Right, Up, or Down. Starting at a specific state, the agent performs multiple runs, collecting rewards for each transition until reaching a terminal state. The utility of the starting state is then calculated as the average total reward across all runs.

For instance:

- Starting at state (1,1), the agent observes total rewards of 0.7, 0.8, and 0.6 over three trials.
- The utility of state (1,1) is estimated as $(U(1,1) = (0.7 + 0.8 + 0.6) / 3 = 0.7)$.

Limitations

Direct Utility Estimation has notable drawbacks:

- **Independence Assumption:** It incorrectly assumes that state utilities are independent, ignoring the Markov property of RL environments.
- **Slow Convergence:** The method requires a large number of trials to produce accurate utility estimates, making it computationally expensive and inefficient.

Adaptive Dynamic Programming in AI

1

Adaptive Dynamic Programming (ADP), also known as Approximate Dynamic Programming, Neuro-Dynamic Programming, and Reinforcement Learning (RL), is a set of techniques used to solve optimal control problems for discrete-time (DT) and continuous-time (CT) nonlinear systems. ADP is particularly useful in scenarios where the system dynamics are complex or partially unknown.

Key Principles

ADP involves iteratively improving the control policy based on the observed performance and system dynamics. It can be applied to both model-based and model-free scenarios. The main goal is to find an optimal policy that minimizes a cost function over time.

Algorithms and Techniques

Several algorithms and techniques fall under the umbrella of ADP, including:

1. **Value Iteration:** This algorithm iteratively updates the value function for DT systems until convergence. It can also be applied with a positive semi-definite initial value function.
2. **Policy Iteration:** This involves iteratively improving the policy for DT and CT systems by evaluating and improving the policy until an optimal policy is found.
3. **Integral Reinforcement Learning:** This technique is used for partially unknown CT systems and involves learning the value function and policy simultaneously.
4. **Model-Free Integral Reinforcement Learning:** This is used for completely unknown nonaffine CT systems and does not require a model of the system dynamics.
5. **Model-Free Nonzero-Sum Games:** This technique is used for completely unknown nonaffine CT systems in a game-theoretic context.
6. **Online Learning Policy Update:** This involves updating the policy for DT systems in an online manner, without requiring a complete model of the system.
7. **Online Learning Without Initial Admissible Control:** This technique is used for CT systems and does not require an initial admissible control policy.
8. **Parallel Control-Based Optimal Tracking:** This involves tracking a desired trajectory for nonaffine CT systems using parallel control strategies.

Practical Applications

ADP has numerous practical applications, including:

- **Robotics:** Optimizing the control policies for robotic systems to achieve desired tasks.
- **Finance:** Developing trading strategies that maximize returns while minimizing risks.
- **Healthcare:** Optimizing treatment plans for patients to improve health outcomes.

- **Energy Systems:** Managing the operation of power grids to ensure efficient and reliable energy distribution.

Important Considerations

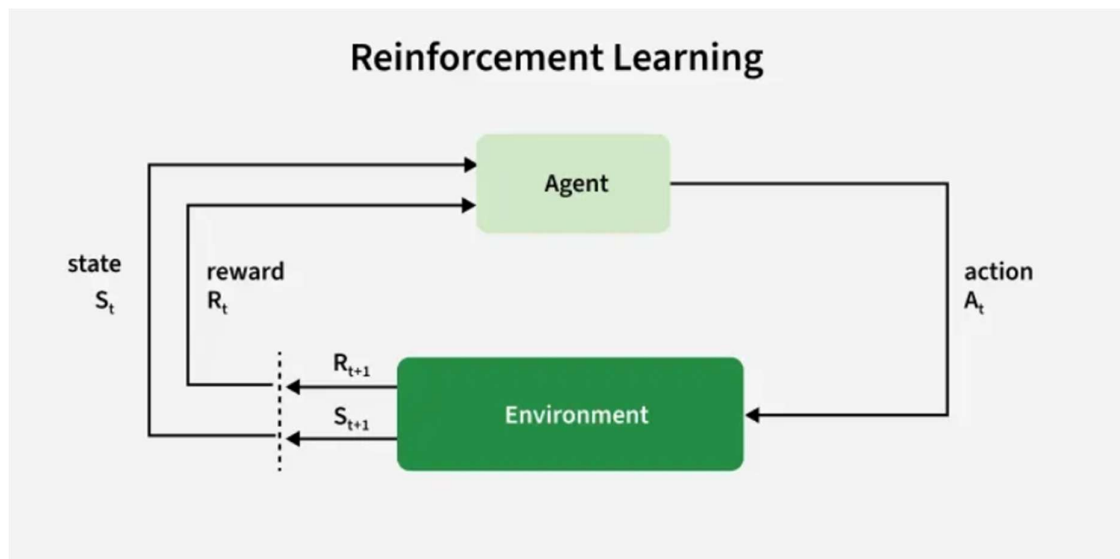
While ADP is a powerful tool, it has some limitations:

- **Computational Complexity:** The algorithms can be computationally intensive, especially for large state and action spaces.
- **Convergence:** Ensuring convergence to an optimal policy can be challenging, particularly in model-free scenarios.
- **Scalability:** Scaling ADP algorithms to real-world problems with high-dimensional state and action spaces can be difficult.

In conclusion, Adaptive Dynamic Programming is a versatile and powerful approach for solving optimal control problems in AI. By iteratively improving the control policy based on observed performance, ADP can handle complex and partially unknown systems effectively.

Temporal Difference (TD) learning

Temporal Difference (TD) Learning is a model-free reinforcement learning method used by algorithms like Q-learning to iteratively learn state value functions ($V(s)$) or state-action value functions ($Q(s,a)$). Unlike Monte Carlo methods that require full episodes, TD Learning updates value estimates after each time step using the Bellman equation and Temporal Difference error.



Reinforcement Learning

How TD Learning Combines Monte Carlo and Dynamic Programming

- **Monte Carlo Methods:** Learn value estimates by averaging returns after complete episodes. They require the episode to finish before updating values, which makes them unsuitable for continuing (non-episodic) tasks and can lead to high variance in updates.
- **Dynamic Programming (DP):** Uses a known model of the environment to perform updates by bootstrapping updating value estimates based on other learned values, not just actual returns. DP requires full knowledge of transition and reward probabilities, which is often impractical.

TD Learning: It uses both approaches

- Like [Monte Carlo](#), TD learning does not need a model of the environment (model-free).
- Like DP, TD learning updates predictions based on other learned predictions (bootstrapping), not just actual returns.
- TD methods can learn after every step, not just at episode end, making them suitable for non-episodic and sequential tasks.

Why TD is Model-Free and Non-Episodic

- **Model-Free:** TD learning does not require knowledge of the environment's transition or reward structure. It learns directly from experience by updating values based on observed rewards and subsequent predictions.

- **Non-Episodic (Sequential):** TD learning can update values after every step, not just after an episode ends. This makes it suitable for tasks that do not have clear episode boundaries, such as real-time control or ongoing games.

Core Components

1. **Bellman Equation:** The foundation for TD updates. For a state value function:
2. **Temporal Difference (TD) Error:** $\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
3. **TD Learning Update Rule:** $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$
4. **Expanded TD Update (in one step):**

How TD Learning Works: Example with Bellman Equation

.

Step 1: Agent Takes an Action

Step 2: Apply the Bellman Equation

Step 3: Calculate Temporal Difference Error

Step 4: Update the Value Function

Q-Learning in Reinforcement Learning

Q-Learning is a popular [model-free reinforcement learning algorithm](#) that helps an agent learn how to make the best decisions by interacting with its environment. Instead

of needing a model of the environment the agent learns purely from experience by trying different actions and seeing their results.

Q-Learning is a **model-free, value-based, off-policy** RL algorithm. It estimates the *quality* (**Q-value**) of taking a given action in a given state, aiming to learn the optimal policy that maximizes future rewards.

Core Concepts:

- **State (S):** Current situation of the agent.
- **Action (A):** Possible move from the current state.
- **Reward (R):** Feedback from the environment.
- **Q-Table:** A matrix storing Q-values for each state-action pair.

\

Key Components

1. Q-Values or Action-Values

Q-values represent the expected rewards for taking an action in a specific state. These values are updated over time using the Temporal Difference (TD) update rule.

2. Rewards and Episodes

The agent moves through different states by taking actions and receiving rewards. The process continues until the agent reaches a terminal state which ends the episode.

3. Temporal Difference or TD-Update

The agent updates Q-values using the formula:

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$$

Where,

- **S** is the current state.
- **A** is the action taken by the agent.
- **S'** is the next state the agent moves to.

- **A'** is the best next action in state S'.
- **R** is the reward received for taking action A in state S.
- **γ (Gamma)** is the discount factor which balances immediate rewards with future rewards.
- **α (Alpha)** is the learning rate determining how much new information affects the old Q-values.

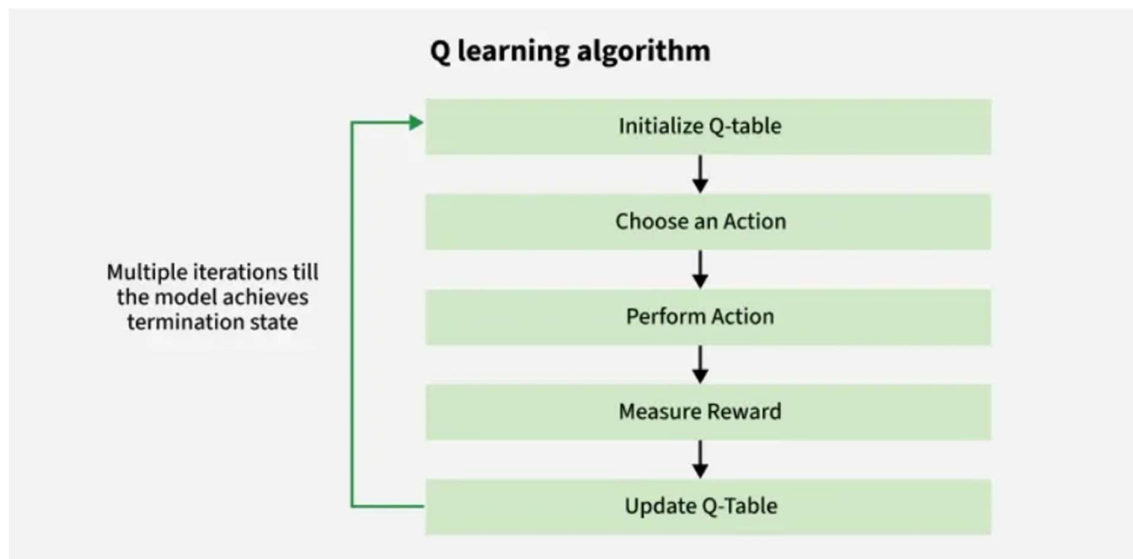
4. ϵ -greedy Policy (Exploration vs. Exploitation)

The ϵ -greedy policy helps the agent decide which action to take based on the current Q-value estimates:

- **Exploitation:** The agent picks the action with the highest Q-value with probability $1 - \epsilon$. This means the agent uses its current knowledge to maximize rewards.
- **Exploration:** With probability ϵ , the agent picks a random action, exploring new possibilities to learn if there are better ways to get rewards. This allows the agent to discover new strategies and improve its decision-making over time.

How does Q-Learning Works?

Q-learning models follow an iterative process where different components work together to train the agent. Here's how it works step-by-step:



Q learning algorithm

1. Start at a State (S)

The environment provides the agent with a starting state which describes the current situation or condition.

2. Agent Selects an Action (A)

Based on the current state and the agent chooses an action using its policy. This decision is guided by a Q-table which estimates the potential rewards for different state-action pairs. The agent typically uses an ϵ -greedy strategy:

- It sometimes explores new actions (random choice).
- It mostly exploits known good actions (based on current Q-values).

3. Action is Executed and Environment Responds

The agent performs the selected action. The environment then provides:

- A **new state (S')** — the result of the action.
- A **reward (R)** — feedback on the action's effectiveness.

4. Learning Algorithm Updates the Q-Table

The agent updates the Q-table using the new experience:

- It adjusts the value for the state-action pair based on the received reward and the new state.
- This helps the agent better estimate which actions are more beneficial over time.

5. Policy is Refined and the Cycle Repeats

With updated Q-values the agent:

- Improves its policy to make better future decisions.
- Continues this loop — observing states, taking actions, receiving rewards and updating Q-values across many episodes.

Over time the agent learns the optimal policy that consistently yields the highest possible reward in the environment.

Methods for Determining Q-values

1. Temporal Difference (TD):

Temporal Difference is calculated by comparing the current state and action values with the previous ones. It provides a way to learn directly from experience, without needing a model of the environment.

2. Bellman's Equation:

Bellman's Equation is a recursive formula used to calculate the value of a given state and determine the optimal action. It is fundamental in the context of Q-learning and is expressed as:

$$Q(s, a) = R(s, a) + \gamma \max_a Q(s', a)$$

Where:

- **Q(s, a)** is the Q-value for a given state-action pair.
- **R(s, a)** is the immediate reward for taking action **a** in state **s**.
- **γ** is the discount factor, representing the importance of future rewards.
- $\max_a Q(s', a)$ is the maximum Q-value for the next state **s'** and all possible actions.

Advantages

- **Trial and Error Learning:** Q-learning improves over time by trying different actions and learning from experience.
- **Self-Improvement:** Mistakes lead to learning, helping the agent avoid repeating them.
- **Better Decision-Making:** Stores successful actions to avoid bad choices in future situations.
- **Autonomous Learning:** It learns without external supervision, purely through exploration.

Disadvantages

- **Slow Learning:** Requires many examples, making it time-consuming for complex problems.
- **Expensive in Some Environments:** In robotics, testing actions can be costly due to physical limitations.
- **Curse of Dimensionality:** Large state and action spaces make the Q-table too large to handle efficiently.
- **Limited to Discrete Actions:** It struggles with continuous actions like adjusting speed, making it less suitable for real-world applications involving continuous decisions.